Online Menu Ordering System

This plan outlines the architecture and implementation steps for building an online menu ordering system with a clean, premium frontend and a Python backend.

User Review Required

> IMPORTANT
> Please review the proposed features and design. Once approved, I will begin the implementation. The backend will use **Flask**. I'll assume you have Flask installed in your Python environment or I can install it if needed. Please let me know if you prefer another framework like FastAPI.

Proposed Architecture & Features

Backend (Python - Flask)

- `App.py`: The main Flask application server.
- **Menu Data**: A hardcoded Python dictionary containing 5 categories (Starters, Main Course, Beverages, Desserts, Breads) with at least 10 items in each.
- **API Endpoints**:
  - `GET /api/menu`: Returns the complete menu data as JSON.
  - `POST /api/bill`: Receives the user's cart (item IDs and quantities) and calculates the final bill on the backend (applying taxes/discounts if any) to return a formal receipt.

Frontend (HTML/CSS/JS)

- `templates/index.html`: The main interface. It will feature a hero section, category filter buttons, a dynamic menu grid, and a sliding shopping cart panel.
- `static/p.css`: The styling sheet. It will employ a modern, premium aesthetic. We will use a sleek dark/light mode combo, glassmorphic elements, vibrant accent colors, and smooth micro-animations on hover and add-to-cart actions.
- `static/app.js`: Client-side logic to fetch the menu, render it dynamically, handle category filtering, track the shopping cart state, and communicate with the backend to generate the final bill.

Proposed Changes

Flask Backend

[MODIFY] App.py

elements, vibrant accent colors, and smooth micro-animations on hover and add-to-cart actions.
- `static/app.js`: Client-side logic to fetch the menu, render it dynamically, handle category filtering, track the shopping cart state, and communicate with the backend to generate the final bill.

Proposed Changes

Flask Backend

[MODIFY] App.py

- Set up the Flask application.
- Define the 50+ menu items categorized properly.
- Add routes for serving the main page, fetching the menu, and calculating the bill.

Frontend Assets

[MODIFY] index.html

- Create semantic HTML structure (header, main, aside for cart).

[MODIFY] p.css

- Add CSS variables for consistent theming.
- Implement responsive grid layouts.
- Add premium visual features (shadows, gradients, animations).

[NEW] app.js

- Fetch menu data from `/api/menu`.
- Implement filtering logic.
- Implement cart management (add, remove, change quantity).
- Submit cart to `/api/bill` and display the generated bill.

Verification Plan

Automated & Manual Verification

- Run the Flask server locally.
- Use the provided browser agent to verify that the menu renders correctly.
- Ensure the filtering works flawlessly.
- Add items to the cart, submit the order, and verify the bill generated by the Python backend is accurate.